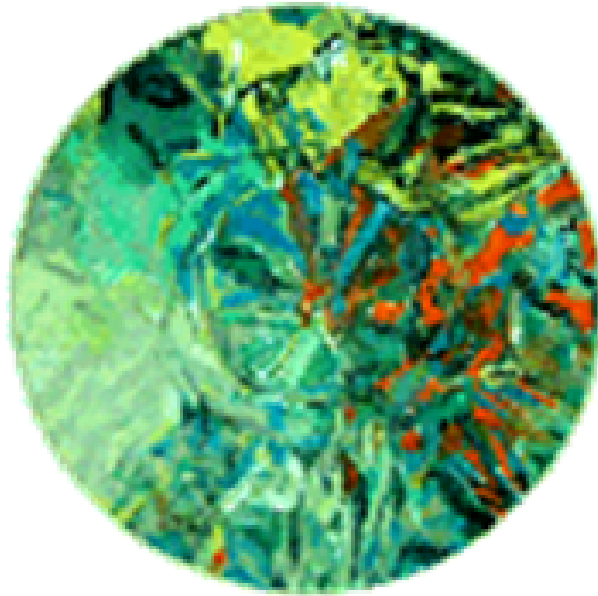




IES CAMAS

MANUEL JESUS DOMINGUEZ DIEGUEZ

PROGRAMACIÓN DE PROCESOS Y SERVICIOS

SOCKETS

Documentación técnica: Ejercicio 3 - Sockets.

Nombre: Manuel Jesús Domínguez Diéguez.

Asignatura: Programación de Procesos y Servicios.

Fecha: 23 de Enero de 2026.

1. Introducción y arquitectura.

En esta práctica he implementado y mejorado un sistema cliente-servidor utilizando sockets en Java. La arquitectura desplegada es distribuida, simulando un entorno de producción real mediante virtualización.

- **Servidor:** ejecutado en un VM (Linux Lite). Gestiona la lógica del juego, genera el número secreto y mantiene el registro de intentos.
- **Cliente:** ejecutado en el sistema anfitrión (host). Actúa como interfaz de usuario, capturando la entrada de la consola y mostrando las respuestas en el servidor.
- **Conectividad:** la comunicación se realiza utilizando el protocolo TCP/IP a través del puerto 7777.

2. Mejoras aplicadas.

Se han realizado 3 modificaciones sobre el código base RandomSocket para mejorar la funcionalidad y la robustez del sistema.

Mejora 1. Protocolo de identificación.

- Antes: el juego iniciaba directamente preguntando por el número.
- Ahora: el servidor solicita el nombre del jugador al conectar y envía un saludo personalizado. Esto no solo mejora la experiencia de usuario sino que sienta las bases para futuras funcionalidades como guardar puntuaciones o registros (logs).
- Beneficio: mejora la experiencia de usuario (UX).

Mejora 2. Contador de intentos.

- Antes: el servidor solo indicaba si el número era mayor o menor. No había registro del progreso.
- Ahora: he añadido una variable en el servidor que cuenta los intentos. Cada vez que el cliente falla en su intento por adivinar el número secreto, el contador aumenta y la respuesta del servidor incluye el número de intento actual.
- Beneficio: ofrece información útil al jugador sobre su rendimiento (feedback) y gamifica la experiencia.

Mejora 3. Validación de entradas y control de errores.

- Antes: el servidor intenta convertir directamente la entrada a entero. Si se introduce una cadena de texto, el servidor envía un mensaje diciendo "introduce un número".

- Ahora: la respuesta del servidor es más clara e informa al usuario de el tipo de dato a introducir.
- Beneficio: aumenta la estabilidad del servidor y la interacción con el usuario evitando errores.

3. Preguntas de comprensión del código.

1. Arquitectura y comunicación.

- **¿Qué papel cumple el servidor y cual el cliente?**

El servidor actúa como el “cerebro” o lógica; gestiona el estado del juego y las reglas. El cliente actúa como una terminal o interfaz encargada únicamente de enviar información del usuario y mostrar la respuesta procesada.

- **¿Qué tipo de comunicación se utiliza (bloqueante/no bloqueante)? ¿Por qué?.**

Se utiliza la comunicación bloqueante (síncrona). Cuando el código ejecuta `entrada.readLine()` el programa se detiene esperando a que lleguen datos por la red.

- **¿Por qué es importante separar el servidor (VM) y el cliente (host)?.**

Porque ejecutar el servidor en la VM obliga a utilizar la pila de red real (TCP/IP) en lugar de la memoria local (localhost). Esto permite entender mejor cómo funcionan realmente las conexiones en la red direccionamiento de IP y puertos, simulando un entorno de producción real.

2. Flujo de ejecución.

- **Describe paso a paso qué ocurre.**

1. El cliente envía un mensaje usando `PrintWriter`.
2. El mensaje viaja por la red y llega al buffer de entrada del servidor.
3. El servidor lee el mensaje con `entrada.readLine()`;
4. El servidor procesa la lógica.
5. El servidor genera y envía una respuesta.
6. El cliente, el cual estaba esperando (bloqueado), recibe la respuesta y la muestra por pantalla.

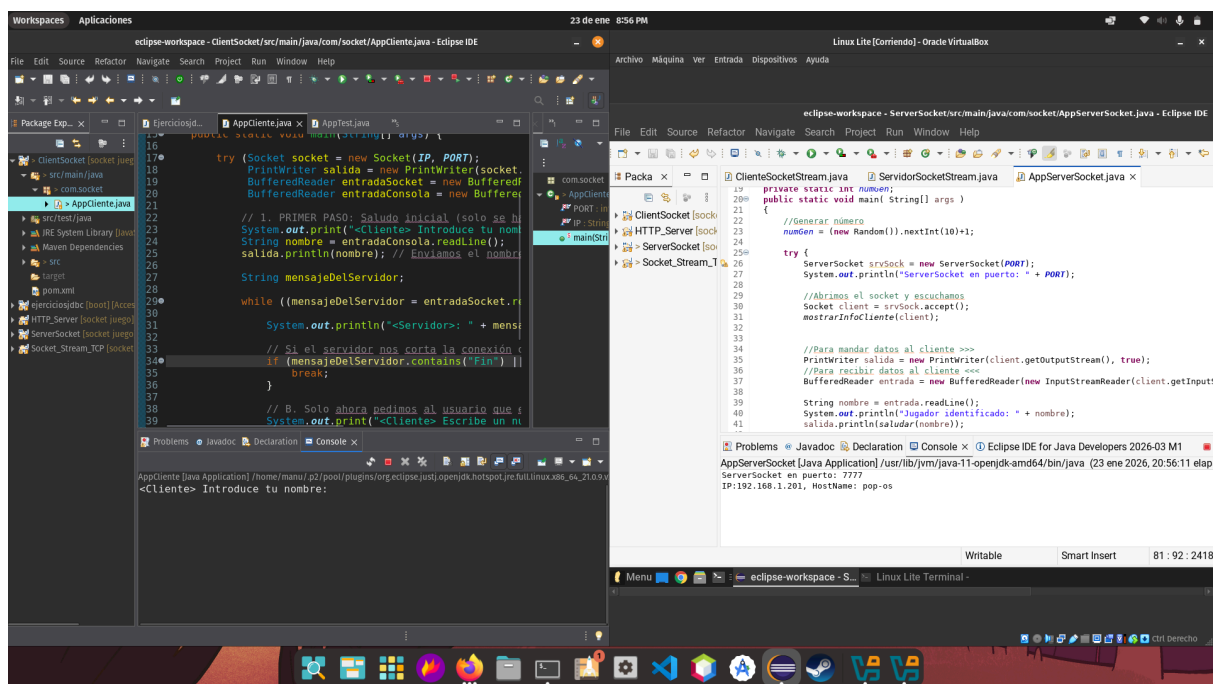
- **¿En qué punto del código se realiza la lectura del mensaje?.**

La lectura se realiza en el bucle, en la instrucción: `while((datoRec = entrada.readLine()) != null)`.

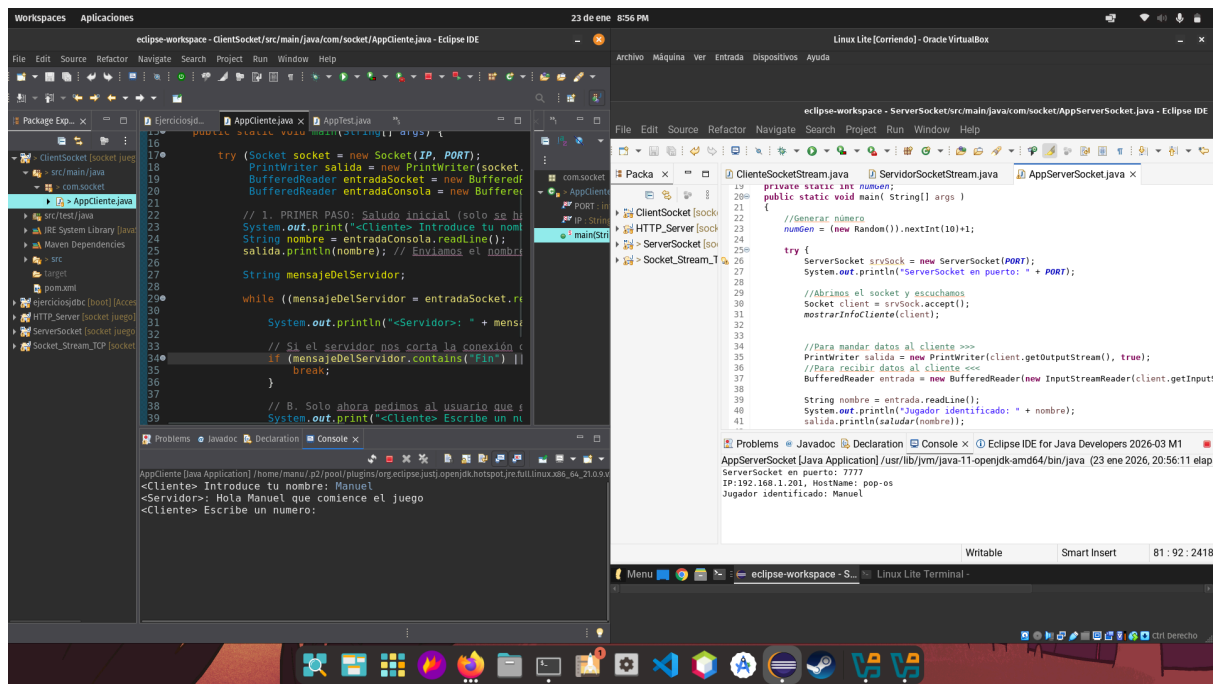
- **¿Dónde se genera y envía la respuesta del servidor?**
La respuesta se genera en el servidor (dentro del método `checkNumero`) y se envía mediante un objeto `PrintWriter` llamada `salida`: `salida.println(datoEnv)`.
- **¿Qué cambios serían necesarios para atender a varios clientes simultáneamente?**
Sería necesario implementar concurrencia (`multithreading`). El hilo principal deja de gestionar la lógica del juego y se limitaría a un bucle infinito aceptando conexiones (`accept()`). Cada vez que un cliente se conecta, se crearía un nuevo hilo (`thread`) dedicado a ese cliente y lo iniciaría (`start()`).

4. Evidencias de ejecución.

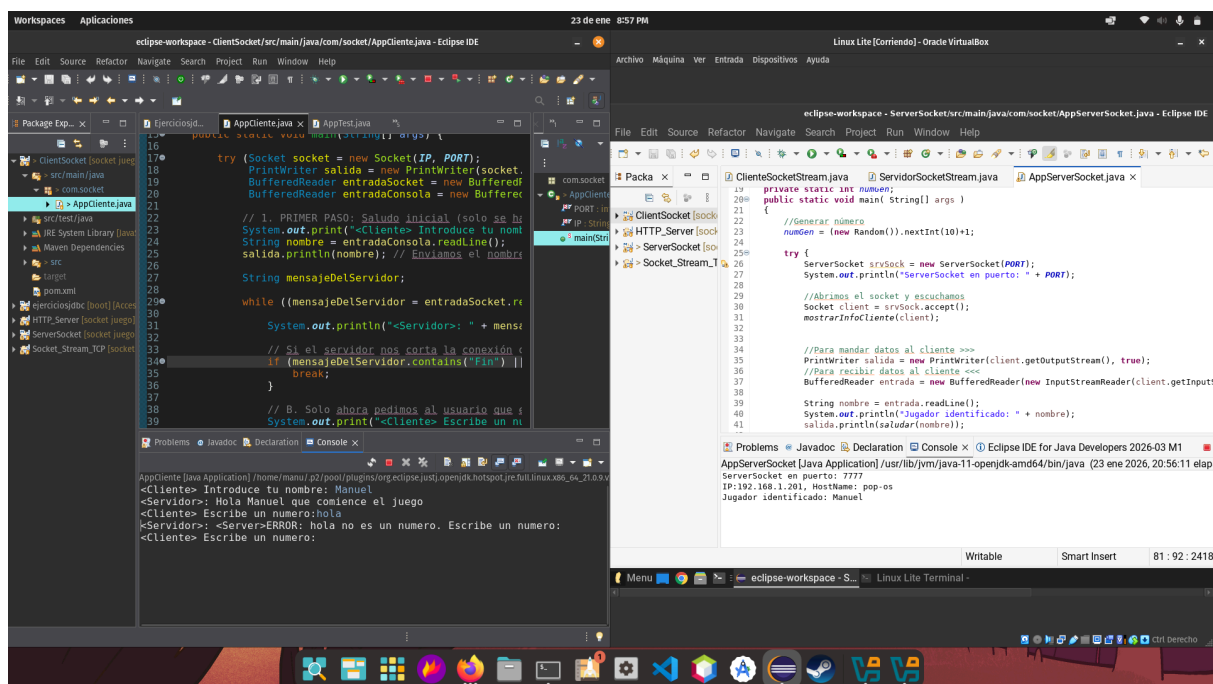
Conexión del cliente y el servidor: el cliente está ejecutando en mi máquina principal (Linux Pop-os) y el servidor en la máquina virtual (Linux Lite), en el puerto Pop-os. Tuve que cambiar el localhost por la dirección ip de mi máquina virtual para poder realizar la conexión.



Saludo por parte del servidor: el cliente introduce su nombre en la terminal y el servidor le devuelve el saludo. Acto seguido comienza el juego.



Validación de entrada de datos: se valida que solo se introduzca un número y el servidor aclara al cliente que solo puede introducir números.



Dinámica de juego: el servidor procesa la información enviada desde el cliente y determina si acierta o no el número. También lleva un conteo del número de intentos que se muestra una vez se adivina el número.

Nota: cuando se adivina el número, el socket se cierra debido al break.

